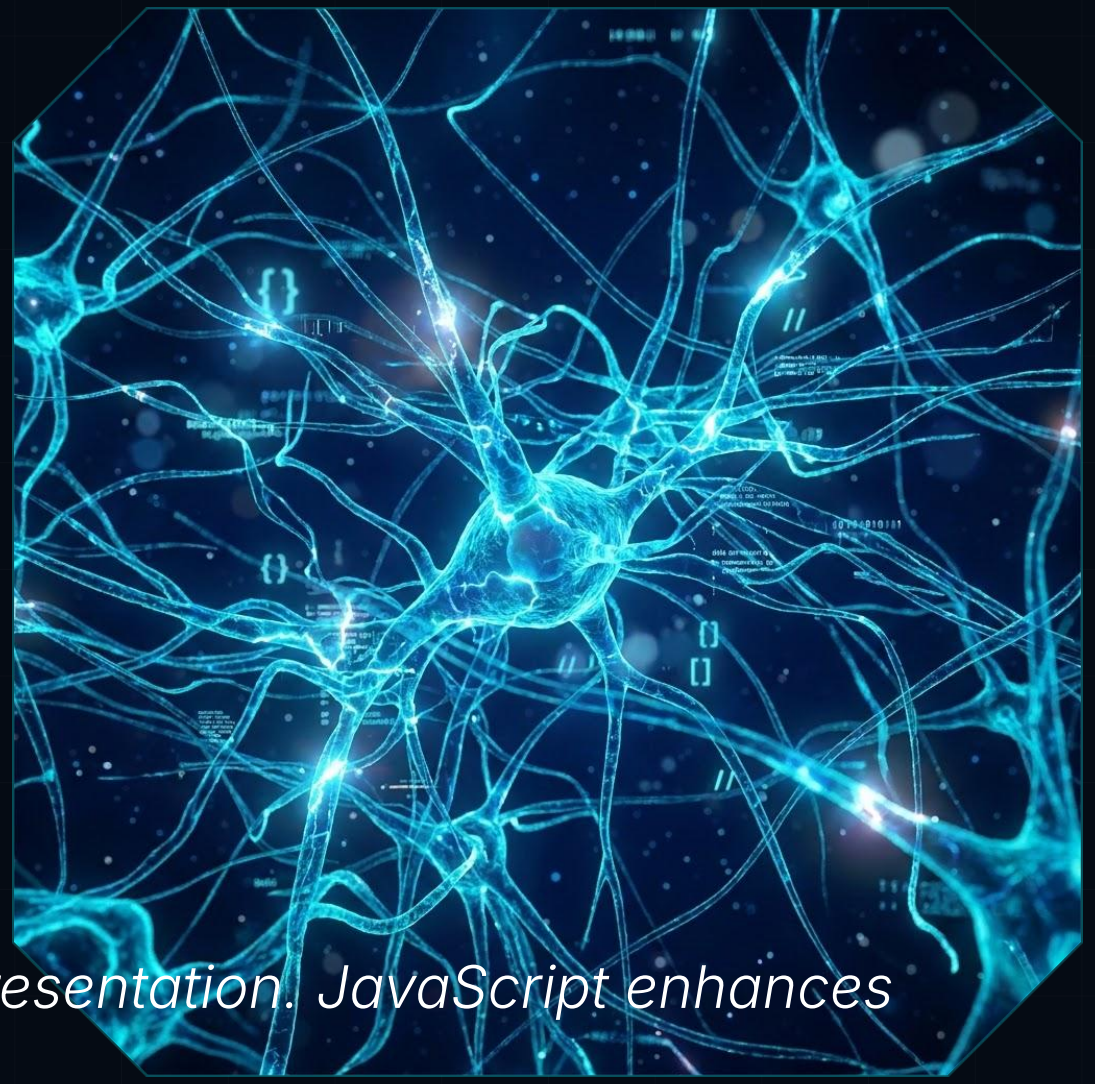
Lab 3: Production React 19, Fiber Hooks & Concurrent UI

From Custom Mini-React Engine (Lab 2) to Production Hooks, Lifecycles



MSc. Trần Vĩnh Khiêm
University of Information Technology, VNUHCM

> *"HTML is the foundation. CSS enhances presentation. JavaScript enhances behavior. AI enhances the developer."*

ARCHITECTURAL CONTINUITY

From Lab 2 Mini-React Engine to Production React 19

Why we built our own VDOM in Lab 2 and how Production React 19 scales those exact primitives.

Bridging Lab 2 Primitives to Lab 3 Production

- **Lab 2 VNode & renderToDOM:** In Lab 2 Sprint 1, you built synchronous createElement() and recursive renderToDOM(). Today we upgrade to React 19 Fiber createRoot().
- **Lab 2 Global Cursor Array:** In Lab 2 Sprint 2, stateStore[cursor] lived in a global array. In React 19, hooks live in a per-Fiber linked list (memoizedState).
- **Lab 2 Event Delegation Hub:** Your custom setupEventDelegation(root) becomes React 19's SyntheticEvent system & Accessible Controlled Forms.
- **Lab 2 ViewState & Anti-Drift:** Your Lab 2 ViewState<T>, AudioTaskQueue, and createAntiDriftTimer evolve into Production Hooks & KRAFTON PUBG VN Telemetry.

lab2-to-lab3-bridge.ts

```
// LAB 2 (What you built from scratch last week):
import { createElement, renderToDOM } from './mini-react.js';
import { useState, resetCursor } from './reactive-engine.js';
// Limitation: Full tree teardown & global cursor collisions!

// [OK] LAB 3 (Production React 19 Fiber Architecture):
import { StrictMode, useState, useEffect, useId } from 'react';
import { createRoot } from 'react-dom/client';

// Per-Fiber isolated hook queues + time-sliced work loop:
const root = createRoot(document.getElementById('root')!);
root.render(
  <StrictMode>
    <PubgTelemetryDashboard />
  </StrictMode>
);
```

PEDAGOGICAL MANIFESTO

Engineering Accountability in the AI Era: Generate & Verify

Continuing Lab 2's core philosophy into complex stateful React component trees.

Foundational Engineering Principles

- **HTML is the Foundation:** JSX compiles to _jsx() calls, but the browser still renders an HTML5 DOM & Accessibility Tree. Zero div-soup allowed.
- **CSS Enhances Presentation:** Scoped CSS Modules and Custom Properties guarantee 375px-to-1440px visual stability with zero layout thrashing.
- **JavaScript & React Govern State:** Pure functional components map immutable state snapshots deterministically to UI (UI = f(state)).
- **AI Enhances, Engineers Verify:** LLMs generate probabilistic React code full of legacy patterns; you are accountable for proving runtime correctness.

react19-verification-contract.ts

```ts
// Engineering Invariant Contract (Lab 2 -> Lab 3):
export interface ComponentContract<TProps, TState> {
  readonly props: Readonly<TProps>;
  readonly state: Readonly<TState>;
  // Every useEffect side-effect MUST return a deterministic cleanup:
  registerEffect(effect: () => () => void): void;
}

// 4-Stage Anti-Monolithic Loop:
// 1. Vertical Slice (TASK_DECOMPOSITION.md)
// 2. Props & State Machine Contract
// 3. Atomic AI Generation (constrained by project-rules.md)
// 4. React DevTools Profiler + Lighthouse Audit + Git Commit
```

DEPRECATION AUDIT (1/2)

Purging Legacy React 16/17 APIs from the 2022 Lab Deck

Auditing Slides 4–25 of the legacy Lab3_React.pdf and enforcing React 19 Baseline standards.

Deprecated Patterns (Strictly Forbidden in Lab 3)

- **BANNED: ReactDOM.render() (Slides 18–25):** Deprecated in React 18 and deleted in React 19. Blocks concurrent time-slicing; use createRoot from 'react-dom/client'.
- **BANNED: class extends React.Component (Slides 7–17, 24):** Legacy lifecycle boilerplate and unbound 'this' bugs. Exercise 4.7's prompt to refactor Function → Class is abolished.
- **BANNED: Component.defaultProps (Slide 13):** Deprecated in React 18.3+ and removed on functional components in React 19. Use ES6 default parameter destructuring.
- **BANNED: import * as React (Slides 4–17):** As learned in Lab 2's Automatic JSX Runtime (_jsx), modern Vite projects only import explicit hooks.
- **BANNED: antd/dist/antd.css & http:// URLs (Slides 19, 21):** Removed in Ant Design v5; insecure HTTP avatar links trigger Mixed Content security errors.

deprecation-audit-part1.jsx

```jsx
// [BAD] LEGACY 2022 LAB DECK (Slides 7, 13, 18 - FORBIDDEN):
import * as React from 'react';
import * as ReactDOM from 'react-dom';
class MyButton extends React.Component {
  render() { return <button>{this.props.text}</button>; }
}
MyButton.defaultProps = { text: 'Click' }; // [BAD] Removed in React 19
ReactDOM.render(<MyButton />, document.getElementById('root'));

// [OK] MODERN REACT 19 BASELINE (Lab 03):
import { createRoot } from 'react-dom/client';
export function MyButton({ text = 'Click', disabled = false }) {
  return <button type='button' disabled={disabled}>{text}</button>;
}
createRoot(document.getElementById('root')).render(<MyButton />);
```

DEPRECATION AUDIT (2/2)

Eliminating Constructor Side Effects, State Mutation & A11y Bugs

Connecting Lab 2's Immutability & Key Heuristics to fix the 5 architectural bugs in Lab3_React.pdf.

Architectural Anti-Patterns Caught in Legacy Deck

- **BANNED: setTimeout in constructor() (Slide 11 - Ex 1.1):** Firing timers in a constructor leaks memory on unmount and double-fires in StrictMode. Must use useEffect + cleanup.
- **BANNED: In-Place State Mutation (Slide 12 - Ex 1.2):** Calling appState.items.push('Fourth') keeps the same memory reference, failing Lab 2's Object.is() state check!
- **BANNED: Primitive/Index Keys (Slide 8 - Ex 0.4):** Using key={i} violates Lab 2's O(n) reconciliation key contract when items duplicate, sort, or delete.
- **BANNED: Triple <h1> for Key-Value Data (Slide 22 - Ex 4.5):** Using three <h1> tags for firstName, lastName, age destroys WCAG heading hierarchy; use <dl>/<dt>/<dd>.
- **BANNED: Blocking window.alert() & keyCode (Slides 19, 26):** alert() freezes the main thread; keypress/keyCode are purged in favor of role='alert' and e.code.

deprecation-audit-part2.jsx

```jsx
// [BAD] LEGACY BUG 1 (Slide 11): Timer inside constructor()
constructor() {
  super();
  setTimeout(() => this.setState({ content: 'Done!' }), 3000);
}
// [BAD] LEGACY BUG 2 (Slide 12): Direct array mutation
appState.items.push('Fourth'); render(appState);

// [OK] MODERN LAB 3 FIX: Leak-free useEffect & Immutable Spread
useEffect(() => {
  const id = window.setTimeout(() => setStatus('Done!'), 3000);
  return () => window.clearTimeout(id);
}, []);
const addItem = (item) => setItems((prev) => [...prev, item]);
```

CURRICULUM ARCHITECTURE

The 3-Tier Knowledge Taxonomy: Core, Awareness, Later

Building systematically on Lab 2's VDOM & State Store foundation.

Cognitive Stratification for Lab 3

- **Tier 1: CORE (Master & Defend Live in Class):** React 19 createRoot, Pure Functional Components, ES6 default props, Semantic JSX (<article>, <dl>), useState immutability, Derived State, useId forms, useEffect cleanup & 4-state UI machines.
- **Tier 2: AWARENESS (Understand for Debugging & Homeworks):** Migrating legacy Class components to Hooks; React.memo / useCallback (from Lab 2's custom memo); Web Audio API polyphony & monotonic clock drift synchronization.
- **Tier 3: LATER LABS (Deferred to Lab 4–6):** Global state managers (Zustand, Redux Toolkit), client/server routing (TanStack Router, Next.js App Router), and React Server Components (RSC).

lab3-curriculum-matrix.json


```
{
 "Lab2_Foundation_Recap": [
  "POJO VNode & O(n) Key Reconciliation",
  "Closure Cursor useState & Root Event Delegation"
 ],
 "Lab3_Tier1_Core": {
  "Mounting": "createRoot ('react-dom/client') + StrictMode",
  "Components": "Pure Functions + ES6 Default Props + Children",
  "Hooks": "useState, useEffect (with cleanup), useId",
  "Resilience": "Finite State Machine + AbortController"
 },
 "Lab3_Tier2_Awareness": [
  "React.memo (88-Key Piano) & Web Audio Polyphony",
  "KRAFTON PUBG VN Anti-Drift Clock Synchronization"
 ]
}
```

TIMEBOXED ROADMAP

180-Minute Lab Execution Plan: 5 Strict Timeboxed Blocks

Structured sprint execution with mandatory DevTools checkpoints and atomic Git commits.

The 5 Execution Blocks (180 Minutes Total)

- **Block 1 (00–40m): Lab 2 Bridge, Audit & AI WBS Setup:** Connect Lab 2's mini-react.js to React 19 Fiber; configure project-rules.md and TASK_DECOMPOSITION.md.
- **Block 2 (40–75m): In-Class Sprint 1 (Checkpoint 1):** Exercise 1: Semantic Engineer & Pro-Gamer Profile Card (<DeveloperCard />). Zero div-soup, <article>/<dl>, 2 Git commits.
- **Block 3 (75–120m): In-Class Sprint 2 (Checkpoint 2):** Exercise 2: Interactive Pioneer & Squad Directory. Derived state (.filter/.toSorted), useId forms, 3 Git commits.
- **Block 4 (120–155m): In-Class Sprint 3 (Checkpoint 3):** Exercise 3: KRAFTON PUBG VN Live Match Telemetry Widget. 4-State Machine, Skeleton UI, AbortController cleanup.
- **Block 5 (155–180m): 3-Minute Live Defense & Homeworks:** Live 60–180s architectural mutation defense at the grading desk (AI closed) + HW1, HW2 & HW3 (PUBG VN Arena) briefing.

sprint-schedule.sh

```
# STRICT 180-MINUTE LAB 3 TIMELINE
[00:00 - 00:40] BLOCK 1: Lab 2->3 Bridge & WBS Init
  └── git commit -m 'chore: init react19 vite & rules'
[00:40 - 01:15] BLOCK 2: Sprint 1 (Exercise 1 - 35m)
  └── Checkpoint 1: Zero <div> & Accessibility Tree Audit
[01:15 - 02:00] BLOCK 3: Sprint 2 (Exercise 2 - 45m)
  └── Checkpoint 2: Derived State & React Profiler Audit
[02:00 - 02:35] BLOCK 4: Sprint 3 (Exercise 3 - 35m)
  └── Checkpoint 3: PUBG VN Telemetry & Slow 3G Abort Test
[02:35 - 03:00] BLOCK 5: 3-Minute Live Refactoring Defense
  └── 1-on-1 Live Code Mutation (AI Tools Strictly Closed)
```

WORKFLOW PROTOCOL

Anti-Monolithic Vibe Coding: project-rules.md & WBS

Enforcing the 4-Stage Contract-First Pipeline established in Lab 2.

Mandatory AI Governance Rules

- **One-Shot Prompting Ban:** Copying an entire Exercise or Homework prompt into ChatGPT/Claude/Cursor yields 0% on the 15% Git & Audit rubric.
- **Stage 1 & 2: Slice & Define Contract First:** Break features into TASK_DECOMPOSITION.md and write Props/State JSDoc or TypeScript contracts before prompting AI.
- **Stage 3: Constrained Atomic Generation:** Feed project-rules.md into your AI assistant so it never outputs Class components, index keys, or redundant useEffects.
- **Stage 4: DevTools Verification & Atomic Commit:** Verify DOM semantics, React Profiler render counts, and 375px responsiveness before each of your 6–8 Git commits.

project-rules.md

```
# UIT S3T - LAB 3 AI GOVERNANCE RULES (.cursorrules)
1. STACK: React 19 + Vite + ES2024 (Builds on Lab 2).
2. NO LEGACY APIS: Ban class extends React.Component,
   ReactDOM.render, defaultProps, keyCode, window.alert.
3. ZERO DIV-SOUP: Use <main>, <section>, <article>,
   <header>, <dl>, <dt>, <dd>, <figure>, <output>.
4. A11Y FORMS: Pair every <input>/<select> with
   <label htmlFor={id}> using React 19 useId().
5. STATE HYGIENE: Immutable updates only. Never sync
   derived filter/sort state inside useEffect.
6. EFFECT CLEANUP: Every timer, listener, or fetch in
   useEffect MUST return a cleanup / AbortController.
```

ENGINE EVOLUTION (LAB 2 → LAB 3)

From Lab 2's Recursive renderToDOM to React 19 Fiber

Why synchronous recursion drops frames on large trees and how Fiber time-slicing solves it.

Why Lab 2's renderToDOM Needed an Upgrade

- **Lab 2's Call-Stack Bottleneck:** In Lab 2, renderToDOM(vNode) recursively traversed every child in a single synchronous call stack. On 5,000 nodes, the browser freezes!
- **Legacy ReactDOM.render (Lab 3 PDF Slide 18):** React 16/17's ReactDOM.render suffered from the same synchronous blocking limitation and is removed in React 19.
- **React 19 Fiber Linked List:** Each Fiber node holds 3 pointers: child, sibling, and return (parent). React can pause work between fibers to let user keystrokes paint!
- **StrictMode Double-Invoke Verification:** Wrapping <App /> in <StrictMode> deliberately runs setup+cleanup twice in Dev to expose memory leaks early.

fiber-vs-lab2-mount.jsx

```
// LAB 2 ENGINE (Synchronous Recursive Mount):
vNode.props.children.forEach((child) =>
  dom.appendChild(renderToDOM(child)) // Cannot pause!
);

// [OK] LAB 3 PRODUCTION REACT 19 (Concurrent Fiber Root):
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { App } from './App.jsx';

const rootEl = document.getElementById('root');
if (!rootEl) throw new Error('Missing #root mount container');

createRoot(rootEl).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

SEMANTIC JSX & LANDMARKS

Semantic JSX & Description Lists: Eliminating Div-Soup

Fixing Legacy Slides 5, 6 & 22 using W3C Landmarks and React Fragments.

Semantic Corrections Over Legacy Lab 3

- **React Fragments (<>...</>) Over Wrapper Divs:** In Lab 2, every VNode needed a single root tag. In React 19, <Fragment> groups sibling nodes without injecting extra DOM wrappers.
- **Fixing Inverted Landmarks (Slide 6 - Ex 0.2):** Slide 6 placed <main> inside <section>. W3C mandates a single top-level <main> containing <section> and <article> landmarks.
- **Fixing Triple <h1> Abuse (Slide 22 - Ex 4.5):** Slide 22 rendered firstName, lastName, age as three <h1> tags! Key-value metadata belongs in a semantic <dl> (<dt> + <dd>).
- **Automatic XSS Escaping (Lab 2 Continuity):** Just like Lab 2's createTextNode, JSX escapes {userInput} strings automatically—never use dangerouslySetInnerHTML.

semantic-landmarks.jsx

```jsx
// [BAD] LEGACY SLIDE 22 (Broken Heading Hierarchy):
<div>
  <h1>first name: {user.firstName}</h1>
  <h1>last name: {user.lastName}</h1>
  <h1>age: {user.age}</h1>
</div>

// [OK] LAB 3 SEMANTIC STANDARD (<article> + <dl>):
export function PlayerMetadata({ user }) {
  return (
    <article aria-labelledby='player-title'>
      <h2 id='player-title'>{user.firstName} {user.lastName}</h2>
      <dl className='kv-grid'>
        <dt>First Name</dt><dd>{user.firstName}</dd>
        <dt>Last Name</dt><dd>{user.lastName}</dd>
        <dt>Age</dt><dd>{user.age}</dd>
      </dl>
    </article>
  );
}
```

COMPONENT ARCHITECTURE

Pure Functional Components & ES6 Default Props Contracts

Replacing Legacy Class Components (Slide 7) and defaultProps (Slide 13) with pure functions.

Contract-First Functional Component Design

- **Continuing Lab 2's Pure Function Model:** In Lab 2 Slide 15, you learned UI = f(props, state). Every component in Lab 3 is a pure JavaScript function returning JSX.
- **Why defaultProps Was Removed in React 19:** Slide 13 taught MyButton.defaultProps = {...}. React 19 removed it for function components because ES6 parameter defaults are native.
- **Composition via children Prop (Slide 7 Upgrade):** Instead of class MySection extends React.Component, destructure { title, children } directly in the function signature.
- **Explicit button type='button' Rule:** Unadorned <button> defaults to type='submit' in HTML forms! Always specify type='button' for action triggers.

functional-contract.jsx

```jsx
// [BAD] LEGACY SLIDES 7 & 13 (Class & defaultProps):
class MyButton extends React.Component {
  render() { return <button>{this.props.children}</button>; }
}
MyButton.defaultProps = { disabled: false };

// [OK] LAB 3 CONTRACT-FIRST FUNCTIONAL COMPONENT:
export function ActionButton({
  label = 'Deploy Squad',
  variant = 'primary',
  disabled = false,
  onAction,
}) {
  return (
    <button
      type='button'
      className={`btn btn--${variant}`}
      disabled={disabled}
      onClick={onAction}
    >
      {label}
    </button>
  );
}
```

RECONCILIATION & KEYS

Collection Mapping: Applying Lab 2's O(n) Key Heuristic

Connecting Lab 2 Slide 13 (Key Stability) to fix Legacy Slide 8 (Ex 0.4) & Slide 20 (Ex 4.3).

Deterministic List & Object Rendering

- **Recap from Lab 2's diffChildren(old, new):** In Lab 2, you saw how index keys corrupt DOM state when Task A is deleted and Task B shifts into Key 0.
- **Fixing Legacy Slide 8 (array.map(i ⇒ key={i})):** Using raw string values as keys crashes reconciliation when two elements share the same text. Always use a stable entity ID.
- **Object.entries() Over Object.keys() (Slide 8):** Instead of Object.keys(obj).map(k ⇒ obj[k]), use ES2017+ Object.entries(obj).map(([key, val]) ⇒ ...) for clean destructuring.
- **Fragment Keys in Description Lists:** When mapping <dt>/<dd> pairs without a wrapper , use <Fragment key={key}> imported from 'react'.

collection-mapping.jsx

```jsx
import { Fragment } from 'react';

// [BAD] LEGACY SLIDE 8 (Fragile Key & Verbose Object.keys):
// {array.map((i) => <li key={i}>{i}</li>)}
// {Object.keys(obj).map((i) => <li key={i}>{obj[i]}</li>)}

// [OK] LAB 3 DETERMINISTIC MAPPING:
export function SquadStats({ members, statsRecord }) {
  return (
    <section aria-label='Squad Overview'>
      <ul>
        {members.map((m) => (
          <li key={m.id}>{m.ign} ({m.role})</li>
        ))}
      </ul>
      <dl>
        {Object.entries(statsRecord).map(([metric, score]) => (
          <Fragment key={metric}>
            <dt>{metric}</dt><dd>{score}</dd>
          </Fragment>
        ))}
      </dl>
    </section>
  );
}
```

IN-CLASS SPRINT 1 | 35 MINUTES

Exercise 1: Semantic Engineer & Pro-Gamer Profile Card

Upgrading Legacy Ex 0.2, 0.3, 4.1, 4.4 & 4.5 while enforcing Lab 2's Zero Div-Soup Mandate.

Task Specifications & Constraints (40–75m)

- **Objective:** Build <DeveloperCard /> and <SkillBadge /> in React 19 using Vite, synthesizing the Elon Musk Card (Ex 4.4) and User Profile (Ex 4.5).
- **Strict Zero Div-Soup Constraint:** ZERO tags permitted inside DeveloperCard.jsx! Use <article>, <header>, <hgroup>, <dl>, <dt>, <dd>, <ul>, <li>, <Fragment>.
- **CSS Modules & 375px Responsive Contract:** Replace Slide 18's inline style={{ background: '#d0f0c0' }} with DeveloperCard.module.css and CSS Custom Properties.
- **HTTPS Security & Image Dimensions:** Replace Slide 21's insecure http:// avatar with HTTPS, explicit width/height (zero CLS), and fallback initials.
- **Atomic Git Commits (2 Commits):** Commit 1: 'feat(ex1): implement semantic DeveloperCard and SkillBadge' | Commit 2: 'style(ex1): add responsive 375px CSS module'.

ex1-verification-audit.sh

```
# CHECKPOINT 1 VERIFICATION SUITE (Minute 75):

# 1. Verify ZERO <div> soup inside component:
grep -n '<div' src/components/DeveloperCard.jsx
# Expected output: 0 matches (Empty!)

# 2. Verify HTTPS-only media origins:
grep -n 'http://' src/components/DeveloperCard.jsx
# Expected output: 0 matches

# 3. Chrome DevTools Inspection:
# - Elements -> Accessibility Tree: article -> header + dl + list
# - Device Toolbar: iPhone SE (375px) -> Zero horizontal scroll
# - Git Log: 2 atomic commits matching TASK_DECOMPOSITION.md
```

SPRINT 1 CODE CONTRACT

Exercise 1 Reference Contract: DeveloperCard.jsx

Pure functional component with ES6 default parameters, Fragment keys, and zero tags.

Key Architectural Highlights

- **Self-Contained <article> Landmark:** Uses aria-labelledby='dev-heading' bound to the <h2> element so screen readers announce the card title clearly.
- **Key-Value Metadata via <dl>:** Replaces Slide 22's triple <h1> bug with Object.entries(metrics) mapped into <Fragment key={k}><dt/><dd/></Fragment>.
- **Reusable <SkillBadge /> Composition:** Renders each skill object { id, name, level } using stable key={s.id} inside an accessible <ul aria-label='Skills'>.
- **Zero Runtime CSS Overhead:** Uses scoped CSS Modules (styles.card, styles.metrics) instead of legacy Ant Design v4 global CSS imports.

src/components/DeveloperCard.jsx

```jsx
import { Fragment } from 'react';
import styles from './DeveloperCard.module.css';

export function DeveloperCard({
  fullName = 'Tran Vinh Khiem',
  role = 'Senior Systems Architect • UIT S3T',
  avatarUrl =
'https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=240',
  metrics = { commits: 512, vdomNodes: 0, a11yScore: 100 },
  skills = [{ id: 'sk-1', name: 'React 19 Fiber', tier: 'Core' }],
}) {
  return (
    <article className={styles.card} aria-labelledby='dev-heading'>
      <header className={styles.header}>
        <img src={avatarUrl} alt={fullName} width='64' height='64' />
        <hgroup><h2 id='dev-heading'>{fullName}</h2><p>{role}</p></hgroup>
      </header>
      <dl className={styles.metrics}>
        {Object.entries(metrics).map(([k, v]) => (
          <Fragment key={k}><dt>{k}</dt><dd>{v}</dd></Fragment>
        ))}
      </dl>
      <ul className={styles.skills} aria-label='Engineered Skills'>
        {skills.map((s) => <li key={s.id}>{s.name} ({s.tier})</li>)}
      </ul>
    </article>
  );
}
```

REACTIVE STATE ENGINE (LAB 2 → LAB 3)

From Lab 2's Global stateStore[cursor] to Per-Fiber Hooks

Understanding why React 19 attaches Hook linked lists to individual Fiber nodes.

How Lab 2's useState Evolves in Production React

- **Recap of Lab 2 Slide 17 (stateStore[cursor]):** In Lab 2, you wrote let stateStore = []; let cursor = 0; where setState checked !Object.is(prev, next) before calling triggerRender().
- **Why Lab 2's Global Array Wasn't Enough:** If Child A unmounts while Child B stays mounted, a single global array shifts every downstream component's cursor index!
- **React 19's Per-Fiber memoizedState Queue:** React stores a linked list of Hook nodes on each component's own Fiber node (fiber.memoizedState), isolating component lifecycles.
- **Why Slide 12's appState.items.push() Fails:** Because React uses the exact same Object.is(prev, next) check you built in Lab 2! Mutating an array in place keeps the same reference.

state-immutability-comparison.js

```
// RECAP FROM YOUR LAB 2 ENGINE (Slide 17):
if (!Object.is(stateStore[currentCursor], resolvedValue)) {
  stateStore[currentCursor] = resolvedValue;
  triggerRender(); // Only fires if reference/value changed!
}

// [BAD] LEGACY LAB 3 SLIDE 12 BUG (Direct Mutation):
items.push('Fourth');
setItems(items); // Object.is(items, items) === true -> NO RENDER!

// [OK] LAB 3 IMMUTABLE UPDATE (New Reference):
setItems((prev) => [...prev, { id: crypto.randomUUID(), label: 'Fourth'
}]);
setItems((prev) => prev.filter((item) => item.id !== targetId));
```

DATA FLOW & DERIVED STATE

Derived State: Avoiding the #1 AI useEffect Anti-Pattern

Connecting Lab 2's Cascading Re-render lesson (Slide 20) to search, filter, and sort pipelines.

Why Syncing Derived State in useEffect Is a Bug

- **What AI Generates by Default:** When asked to filter a list, AI often creates two useStates (items and filteredItems) and syncs them inside useEffect.
- **Double Render Cascade (Lab 2 Slide 20):** Every keystroke triggers Render #1 (stale filteredItems), runs useEffect, calls setFilteredItems, and forces Render #2!
- **Compute Derived Data Purely During Render:** Filter and sort directly in the render body: const visible = items.filter(...).toSorted(...). Zero extra state, single render pass!
- **ES2023 .toSorted() Over Mutating .sort():** Array.prototype.sort() mutates the array in place! Always use immutable .toSorted((a, b) ⇒ ...) in React.

derived-state-pattern.jsx

```jsx
// [BAD] AI ANTI-PATTERN (Redundant State + Double Render):
const [people, setPeople] = useState(INITIAL_DATA);
const [filtered, setFiltered] = useState(INITIAL_DATA);
useEffect(() => {
  setFiltered(people.filter((p) => p.iq >= minIq)); // [BAD] BAD!
}, [people, minIq]);

// [OK] LAB 3 ENGINEERING STANDARD (Pure Derived State):
const [people, setPeople] = useState(INITIAL_DATA);
const [minIq, setMinIq] = useState(160);
const [sortBy, setSortBy] = useState('iq-desc');

// Computed in 1 pass during render with immutable .toSorted():
const visiblePeople = people
  .filter((p) => p.iq >= minIq)
  .toSorted((a, b) => (sortBy === 'iq-desc' ? b.iq - a.iq : a.age - b.age));
```

EVENT DELEGATION & A11Y FORMS

From Lab 2's Event Delegation Hub to Controlled React Forms

Upgrading Legacy Slides 15–17 with React 19 useId() and numeric type safety.

Fixing Legacy Slides 15, 16, 17 & 25

- **Lab 2 Root Delegation Connection:** In Lab 2 Slide 18, you attached one listener on rootContainer and traversed target.__vnode. React 19 does this automatically!
- **Fixing Orphan <input> Elements (Slides 15–17):** Legacy Exercises 2.2 & 3 rendered raw <input /> tags with no <label>. Use React's useId() hook to bind <label htmlFor={id}>.
- **Fixing String Coercion on Number Inputs (Slide 15):** e.target.value is always a string ('35' + 1 === '351'). Read e.target.valueAsNumber with Number.isNaN fallback.
- **Eliminating Class Method 'this' Loss (Slide 17):** Pure function closures capture state safely without constructor this.onChange = this.onChange.bind(this).

accessible-controlled-form.jsx

```jsx
import { useId, useState } from 'react';

export function FilterControls({ query, minIq, onQueryChange, onIqChange }) {
  const searchId = useId();
  const iqId = useId();

  const handleIqInput = (e) => {
    const val = e.target.valueAsNumber;
    onIqChange(Number.isNaN(val) ? 0 : val);
  };

  return (
    <form role='search' onSubmit={(e) => e.preventDefault()}>
      <label htmlFor={searchId}>Search Name</label>
      <input id={searchId} type='search' value={query}
             onChange={(e) => onQueryChange(e.target.value)} />
      <label htmlFor={iqId}>Minimum IQ</label>
      <input id={iqId} type='number' value={minIq} onChange={handleIqInput}
/>
    </form>
  );
}
```

IN-CLASS SPRINT 2 | 45 MINUTES

Exercise 2: Interactive Scientist & Pioneer Directory

Upgrading Legacy Ex 0.4, 2.2, 3 & 4.3 (smartPeople) using Lab 2's Reconciliation & Derived State.

Task Specifications & Constraints (75–120m)

- **Objective:** Transform Slide 20's static smartPeople array (Goethe, Einstein, da Vinci, Newton, Maxwell, etc.) into a reactive data explorer.
- **Interactive Feature Contract:** 1) Case-insensitive name search, 2) Min-IQ numeric filter (valueAsNumber), 3) Sort by IQ/Age via .toSorted(), 4) Delete & Reset.
- **Strict Derived-State & Accessibility Contract:** Zero useEffect allowed for filtering/sorting! All inputs bound via <label htmlFor> + useId(); result count in role='status' aria-live='polite'.
- **3 Mandatory Atomic Git Commits:** Commit 1: 'feat(ex2): normalize smartPeople schema with stable IDs' | Commit 2: 'feat(ex2): add search, IQ filter & toSorted' | Commit 3: 'feat(ex2): add delete/reset & live status'.

ex2-checkpoint2-verify.jsx

```
// CHECKPOINT 2 DATA SCHEMA & DERIVED PIPELINE:
const SMART_PEOPLE = [
  { id: 'sp-1', name: 'Johann Goethe', age: 82, iq: 210 },
  { id: 'sp-2', name: 'Albert Einstein', age: 76, iq: 205 },
  { id: 'sp-3', name: 'Leonardo da Vinci', age: 67, iq: 180 },
  { id: 'sp-4', name: 'Isaac Newton', age: 84, iq: 190 },
  { id: 'sp-5', name: 'James Maxwell', age: 48, iq: 190 },
  { id: 'sp-6', name: 'William Sidis', age: 46, iq: 200 },
];

// Verify in React DevTools Profiler:
// Typing 1 character in search input causes EXACTLY 1 render pass.
// Deleting 'Johann Goethe' preserves focus and remaining item keys.
```

SPRINT 2 CODE CONTRACT

Exercise 2 Reference Contract: ScientistDirectory.jsx

Complete unidirectional data flow with useId(), valueAsNumber, and ES2023 .toSorted().

Engineering Verification Points

- **Single Source of Truth:** Only raw independent inputs (people, query, minIq, sortBy) live in useState. visiblePeople is calculated on the fly.
- **Type-Safe Numeric Filtering:** Uses e.target.valueAsNumber so minIq remains a number and never silently breaks >= comparisons.
- **Immutable Deletion & Restoration:** handleDelete uses prev.filter(p ⇒ p.id !== id); handleReset restores SMART_PEOPLE snapshot cleanly.
- **Screen-Reader Live Counter:** The <output role='status' aria-live='polite'> element announces filtered counts automatically.

src/components/ScientistDirectory.jsx

```
import { useId, useState } from 'react';

export function ScientistDirectory({ initialData = SMART_PEOPLE }) {
  const qId = useId(); const iqId = useId();
  const [people, setPeople] = useState(initialData);
  const [query, setQuery] = useState('');
  const [minIq, setMinIq] = useState(160);
  const [sortBy, setSortBy] = useState('iq-desc');

  const visible = people
    .filter((p) => p.name.toLowerCase().includes(query.trim().toLowerCase()) && p.iq >=
minIq)
    .toSorted((a, b) => (sortBy === 'iq-desc' ? b.iq - a.iq : a.age - b.age));

  return (
    <section aria-label='Pioneer Directory'>
      <form role='search' onSubmit={(e) => e.preventDefault()}>
        <label htmlFor={qId}>Name</label>
        <input id={qId} type='search' value={query} onChange={(e) =>
setQuery(e.target.value)} />
        <label htmlFor={iqId}>Min IQ</label>
        <input id={iqId} type='number' value={minIq}
              onChange={(e) => setMinIq(Number.isNaN(e.target.valueAsNumber) ? 0 :
e.target.valueAsNumber)} />
      </form>
      <output role='status' aria-live='polite'>Found {visible.length} pioneers</output>
      <ul>{visible.map((p) => <li key={p.id}>{p.name} — IQ {p.iq}</li>)}</ul>
    </section>
  );
}
```

EFFECT LIFECYCLE & CLEANUP

Synchronizing External Systems with useEffect & Cleanup

Replacing Slide 11's leaky constructor setTimeout and connecting Lab 2's AbortController.

The Laws of useEffect Synchronization

- **Why Slide 11's constructor Timer Is Broken:** In Legacy Ex 1.1, setTimeout inside constructor() cannot be canceled if the user unmounts the view before 3000ms.
- **useEffect Runs After Commit Phase:** Wait until React commits the DOM patch (Lab 2's renderToDOM phase), then synchronize timers, network fetches, or WebSockets.
- **Deterministic Cleanup Contract:** Every effect that subscribes to an external resource MUST return a cleanup function: () ⇒ { clearTimeout(id); controller.abort(); }.
- **Dependency Array Honesty:** Every reactive value read inside useEffect must be declared in [deps] so stale closures never occur.

effect-cleanup-upgrade.jsx

```
// [BAD] LEGACY SLIDE 11 (Memory Leak in Constructor):
class MyComponent extends React.Component {
  constructor() {
    super();
    setTimeout(() => this.setState({ content: 'Done!' }), 3000);
  }
}

// [OK] LAB 3 LEAK-FREE HOOK WITH CLEANUP:
import { useEffect, useState } from 'react';

export function WarmupBanner({ delayMs = 3000 }) {
  const [isReady, setIsReady] = useState(false);
  useEffect(() => {
    const timerId = window.setTimeout(() => setIsReady(true), delayMs);
    return () => window.clearTimeout(timerId); // [OK] Cancels on unmount
  }, [delayMs]);
  return <p role='status'>{isReady ? 'System Ready!' : 'Warming up...'}</p>;
}
```

RESILIENT STATE MACHINES

Evolving Lab 2's ViewState<T> with AbortController

Connecting Lab 2 Slide 24 (IDLE | LOADING | SUCCESS | ERROR) to race-condition-proof effects.

Eliminating Async Race Conditions in React 19

- **Recap of Lab 2 Sprint 3 (ViewState<T>):** In Lab 2 Slide 24, you defined status: 'IDLE' | 'LOADING' | 'SUCCESS' | 'ERROR' instead of boolean soup (isLoading && isError).
- **The Missing Piece in Lab 2: Stale Network Responses:** What happens if the user switches from Squad A to Squad B while Squad A's fetch is still in flight? Squad A overwrites Squad B!
- **AbortController inside useEffect:** Instantiate new AbortController() inside useEffect, pass { signal } to fetch, and call controller.abort() in the cleanup return.
- **Dimension-Matched Skeleton Loader:** Render an aria-busy='true' shimmer placeholder during 'LOADING' to guarantee 0.00 Cumulative Layout Shift (CLS).

abortable-state-machine.ts

```
// UPGRADING LAB 2 SLIDE 24 ViewState<T> WITH ABORT SIGNAL:
useEffect(() => {
  const controller = new AbortController();
  setState({ status: 'LOADING', data: null, error: null });

  fetchPubgSquadTelemetry(squadId, { signal: controller.signal })
    .then((data) => setState({ status: 'SUCCESS', data, error: null }))
    .catch((err) => {
      // [OK] Critical: Do not trigger ERROR state on intentional abort!
      if (err.name === 'AbortError') return;
      setState({ status: 'ERROR', data: null, error: err.message });
    });

  // [OK] Cancels in-flight HTTP request when squadId changes or
unmounts:
  return () => controller.abort();
}, [squadId, retryCount]);
```

IN-CLASS SPRINT 3 | 35 MINUTES

Exercise 3: KRAFTON PUBG Vietnam Series (PVS) Live Telemetry

Building a fault-tolerant live esports radar for Vietnamese PUBG pro squads (120–155m).

KRAFTON x PUBG Vietnam Match Telemetry Spec

- **Domain Context (KRAFTON & PUBG VN):** Build <PubgMatchTelemetry /> tracking live PVS stats for Vietnam's top squads: Cerberus Esports (CES), The Expendables (TE - PGC World Champions), and Genius Esports.
- **4-State Machine & Skeleton Radar:** Manage 'IDLE' | 'LOADING' | 'SUCCESS' | 'ERROR' states. During 'LOADING', display a 220px pulsing Blue-Zone Skeleton card.
- **Simulated Packet Loss & AbortController:** Simulate 35% Erangel/Miramar satellite packet loss. Switching squads rapidly must abort stale requests cleanly via AbortController.
- **Zero window.alert() Mandate (Slide 19 Fix):** Render packet timeout errors inside an accessible <section role='alert'> with a 'Reconnect Satellite (Retry)' button.

pubg-vn-telemetry-spec.js

```
// PUBG VIETNAM SERIES (PVS) SQUAD TELEMETRY DATASET:
export const PUBG_VN_SQUADS = [
  { id: 'ces', name: 'Cerberus Esports', map: 'Erangel', phase: 'Phase 6',
    alive: 4, kills: 14, wwcdProb: '68%', status: 'In Safe Zone' },
  { id: 'te',  name: 'The Expendables',  map: 'Miramar', phase: 'Phase 7',
    alive: 4, kills: 18, wwcdProb: '82%', status: 'Holding High Ground' },
  { id: 'ge',  name: 'Genius Esports',   map: 'Taego',   phase: 'Phase 5',
    alive: 3, kills: 9,  wwcdProb: '41%', status: 'Rotating Blue Zone' },
];

// Checkpoint 3 DevTools Audit (Minute 155):
// 1. Network Slow 3G -> Verify 0.00 CLS Skeleton loader
// 2. Rapidly click CES -> TE -> GE -> Verify (canceled) requests
```

SPRINT 3 CODE CONTRACT

Exercise 3 Reference Contract: PubgMatchTelemetry.jsx

KRAFTON PUBG VN live match radar with 4-state machine, AbortController, and retry recovery.

Key Engineering Mechanics in Exercise 3

- **Squad Selector Triggers Effect Cleanup:** Clicking 'The Expendables' while 'Cerberus Esports' is loading immediately invokes controller.abort() on the previous effect.
- **Safe AbortError Guard:** Checks if (err.name === 'AbortError') return; so rapid squad switching never flashes a false error banner.
- **Retry Counter Trigger:** Clicking 'Reconnect Telemetry' increments retryTick, cleanly re-running the useEffect pipeline.
- **2 Mandatory Atomic Git Commits:** Commit 1: 'feat(ex3): implement PUBG VN 4-state telemetry & AbortController' | Commit 2: 'feat(ex3): add skeleton UI & retry alert'.

src/components/PubgMatchTelemetry.jsx

```
import { useEffect, useState } from 'react';

export function PubgMatchTelemetry({ squadId = 'te' }) {
  const [view, setView] = useState({ status: 'IDLE', data: null, error: null });
  const [retryTick, setRetryTick] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setView({ status: 'LOADING', data: null, error: null });
    fetchKraftonPubgTelemetry(squadId, { signal: controller.signal })
      .then((payload) => setView({ status: 'SUCCESS', data: payload, error: null }))
      .catch((err) => {
        if (err.name === 'AbortError') return;
        setView({ status: 'ERROR', data: null, error: err.message });
      });
    return () => controller.abort();
  }, [squadId, retryTick]);

  if (view.status === 'LOADING') return <section aria-busy='true'
className='skeleton'>Syncing KRAFTON Radar...</section>;
  if (view.status === 'ERROR') return (
    <section role='alert'>
      <p>Blue-Zone Link Lost: {view.error}</p>
      <button type='button' onClick={() => setRetryTick((t) => t + 1)}>Reconnect
Satellite</button>
    </section>
  );
  return <article aria-live='polite'><h3>{view.data?.name} — Kills:
{view.data?.kills}</h3></article>;
}
```

EXAMINATION & DEFENSE (155–180M)

The 3-Minute Live Defense Protocol: Architecture Refactoring

Continuing Lab 2's 1-on-1 live examination standard at the grading desk (25% of Lab Grade).

Rules of the 3-Minute Live Defense

- **All AI Assistants Strictly Closed:** Close ChatGPT, Claude, Cursor Composer, and GitHub Copilot before calling the instructor to your desk.
- **Step 1: Git History & WBS Audit (30s):** Instructor runs git log --oneline --graph to verify 6–8 atomic commits matching your TASK_DECOMPOSITION.md.
- **Step 2: DevTools & 375px Inspection (30s):** Zero Console warnings, zero soup in Exercise 1, 100% <label> bindings in Exercise 2, and clean 375px mobile layout.
- **Step 3: Live Architectural Mutation (60–180s):** Instructor modifies one requirement on your live code. You have 180 seconds to refactor and verify it in browser.

live-defense-challenges.sh

```
# LAB 3 LIVE DEFENSE MUTATION CHALLENGES (60-180 SECONDS):

# Challenge A (Ex 2 - Derived State):
# 'Add a Watchlist toggle star to each scientist/player and a
#  Show Watchlisted Only checkbox without adding a useEffect.'

# Challenge B (Ex 3 - KRAFTON PUBG Telemetry):
# 'Add a 5-second auto-polling setInterval to PubgMatchTelemetry
#  with a Pause Radar toggle button and proper cleanup.'

# Challenge C (Ex 1 + Ex 2 - Lifting State Up):
# 'Lift selectedId to <App /> so clicking a row in Exercise 2
#  updates the featured <DeveloperCard /> in Exercise 1.'
```

SECURITY & CODE INTEGRITY

AI Failure Mode Audit: Catching React Hallucinations (15%)

Documenting, proving via DevTools, and manually fixing 3 AI-generated bugs in AI_FAILURE_AUDIT.md.

Top AI Failure Modes in React 19 & Hooks

- **1. Redundant useEffect for Derived State:** AI creates extra useState + useEffect for filtered lists, doubling render passes and violating Lab 2's render cascade rule.
- **2. Stale Closures in Event & Timer Hooks:** AI binds window.addEventListener('keydown', fn) or setInterval(fn, 1000) with [] deps, freezing stale state snapshots.
- **3. Missing AbortController / Audio Cleanup:** AI fires fetch() or creates AudioContext oscillators without aborting or disconnecting nodes on unmount.
- **4. Naive setInterval Countdown Drift:** AI writes setSeconds(s ⇒ s - 1) for PUBG Airdrop timers, drifting by minutes when browser tabs are backgrounded!

AI_FAILURE_AUDIT.md

```
# MANDATORY DELIVERABLE: AI_FAILURE_AUDIT.md (Minimum 3 Bugs)

## Failure #01: Stale Closure in PUBG Live Auto-Polling
- **Prompt**: 'Add 3s auto-refresh to PubgMatchTelemetry'
- **AI Code**: useEffect(() => { setInterval(() => fetch(squadId), 3000)
}, [])
- **Bug**: Empty [] dependency freezes initial squadId ('ces') forever
  and leaks intervals on every squad switch!
- **DevTools Proof**: Network tab showed 'ces' still polling after
  switching UI to 'The Expendables (te)'.
- **Engineering Fix (Commit 9f4c2a1)**: Added [squadId, isPaused]
  dependencies and returned () => clearInterval(id).
```

HOMEWORK 1 | 1 WEEK

HW 1: Production Studio Drum Kit (WCAG 2.2 AA & 100 Lighthouse)

Modernizing Legacy Slide 26 (Drum Kit w/a/s/d/j/k/l) using Lab 2's Event & A11y Standards.

Project Specification & Criteria

- **Connecting Lab 2 HW 1 (WCAG 2.2 AA & 100 LH):** Apply your WCAG 2.2 AA keyboard & focus engineering from Lab 2 HW 1 to refactor Legacy Slide 26's Drum Kit into a production studio app.
- **Physical Key Codes (e.code) Over Deprecated keyCode:** Listen to window 'keydown'/'keyup' using e.code ('KeyW', 'KeyA', 'KeyS', 'KeyD', 'KeyJ', 'KeyK', 'KeyL') so Vietnamese IME (UniKey) never breaks triggers!
- **Anti-Repeat Guard & Input Focus Isolation:** Block OS key-hold spam via if (e.repeat) return; and ignore shortcuts when typing inside <input> elements.
- **Deaf / Hard-of-Hearing Visual Hit Log:** Render an aria-live='polite' <output> hit history and visual decibel meter alongside <button aria-pressed={isActive}> pads.

hw1-drum-kit-contract.jsx

```jsx
import { useEffect } from 'react';

export function useDrumKeyboard(padMap, onTriggerPad, isEnabled =
true) {
  useEffect(() => {
    if (!isEnabled) return undefined;
    const onKeyDown = (e) => {
      if (e.repeat) return; // [OK] Prevent OS auto-repeat flood
      if (['INPUT', 'TEXTAREA', 'SELECT'].includes(e.target.tagName))
return;
      const pad = padMap.get(e.code); // [OK] W3C e.code ('KeyW'), never
keyCode
      if (pad) { e.preventDefault(); onTriggerPad(pad.id); }
    };
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, [padMap, onTriggerPad, isEnabled]);
}
```

HOMEWORK 2 | 1 WEEK

HW 2: 88-Key Polyphonic Piano & Concurrent FIFO Sequencer

Integrating Legacy Slide 27 (88-Key Piano) with Lab 2's memo() & Concurrent AudioTaskQueue.

Project Specification & Criteria

- **Connecting Lab 2 Slide 21 (memo) & HW 2 (FIFO Queue):** Combine Legacy Slide 27's 88-key piano with Lab 2's shallowEqual memoization and Lab 2 HW 2's FIFO AudioTaskQueue!
- **Algorithmic 88-Key Generation (A0 to C8):** Mathematically generate 52 white keys and 36 black keys across 7+ octaves using f = 440 * 2^((midi - 69) / 12). Hardcoding 88 tags is banned.
- **8-Voice Polyphony & FIFO Voice Stealing:** Play chords up to 8 simultaneous Web Audio oscillators. On the 9th concurrent note, steal the oldest voice in FIFO order to prevent clipping.
- **React.memo Render Isolation (88 Components):** Pressing 1 key must re-render ONLY that key (verified in React DevTools Profiler), never all 88 keys!

hw2-polyphonic-piano-contract.js

```
// BRIDGING LAB 2 HW2 (FIFO Queue) + LAB 2 SLIDE 21 (React.memo):
import { memo } from 'react';

// [OK] Memoized Key: Only re-renders when its own boolean isPressed
flips!
export const PianoKey = memo(function PianoKey({ midi, noteName,
isBlack, isPressed, onNoteOn, onNoteOff }) {
  return (
    <button
      type='button'
      className={`key ${isBlack ? 'key--black' : 'key--white'}`}
      aria-pressed={isPressed}
      onPointerDown={() => onNoteOn(midi)}
      onPointerUp={() => onNoteOff(midi)}
    >
      <span>{noteName}</span>
    </button>
  );
});
```

HOMEWORK 2 ARCHITECTURE

HW 2 Engine: Polyphonic Voice Pool & Async FIFO Replay

Deterministic voice management and abortable asynchronous melody playback.

Concurrency & Memory Hygiene Rules

- **O(1) Active Voice Tracking via ES6 Map:** ES6 Map preserves insertion order (FIFO). When activeVoices.size >= 8, activeVoices.keys().next().value yields the oldest note to release.
- **Click-Free ADSR Envelope Ramp:** Use gain.exponentialRampToValueAtTime(0.0001, ctx.currentTime + 0.04) before osc.stop() to eliminate audio popping noises.
- **FIFO Recording & Variable-Speed Replay:** Record { midi, freqHz, durationMs, deltaMs } into a FIFO queue and replay asynchronously at 0.5x, 1.0x, or 2.0x speed.
- **AbortController Cancellation on Stop/Unmount:** Clicking 'Stop Sequence' immediately cancels all scheduled playback promises and silences active oscillators.

src/audio/PolyphonicSynth.js

```
export class PolyphonicSynth {
  constructor(maxVoices = 8) {
    this.ctx = null;
    this.maxVoices = maxVoices;
    this.voices = new Map(); // FIFO insertion order: midi -> { osc, gain }
  }
  noteOn(midi, freqHz) {
    if (!this.ctx) this.ctx = new window.AudioContext();
    if (this.voices.has(midi)) return;
    if (this.voices.size >= this.maxVoices) {
      const oldestMidi = this.voices.keys().next().value;
      this.noteOff(oldestMidi); // [OK] FIFO voice stealing
    }
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();
    osc.frequency.setValueAtTime(freqHz, this.ctx.currentTime);
    gain.gain.setValueAtTime(0.2, this.ctx.currentTime);
    osc.connect(gain).connect(this.ctx.destination);
    osc.start();
    this.voices.set(midi, { osc, gain });
  }
}
```

HOMEWORK 3 | 1 WEEK (VN ESPORTS)

HW 3: KRAFTON x PUBG Vietnam Championship & Airdrop Arena

Evolving Lab 2 HW 3's Anti-Drift Timer & Virtual List into a Full React 19 Esports Platform.

KRAFTON x PUBG Vietnam Scenario & Requirements

- **Bridging Lab 2 HW 3 to KRAFTON PUBG Vietnam:** Upgrade your Lab 2 HW 3 createAntiDriftTimer and Slide 22 Virtual Scroller into the official 'PUBG Vietnam Series (PVS) Finals Ticket & G-Coin Contraband Crate Arena'.
- **1. Zero-Drift Airdrop & Ticket Countdown Hook:** Convert Lab 2's Date.now() delta timer into usePubgAirdropTimer(targetEpochMs) with visibilitychange resync so background tabs never drift!
- **2. Optimistic Crate/Ticket Claim & 409 Rollback:** Support claiming PVS Grand Finals VIP Tickets (formatted in VND ₫) & G-Coin Crates with optimistic stock deduction and automatic rollback on 409 Conflict.
- **3. Live PUBG VN Leaderboard & Kill-Feed Filter:** Filter & sort Vietnamese squads (The Expendables, Cerberus Esports, Roy Esports, AGG) by WWCD, Placement Points & Kill Points + submit AI_FAILURE_AUDIT.md.

pubg-vn-arena-contract.ts

```ts
// KRAFTON x PUBG VIETNAM ARENA DOMAIN CONTRACT:
export type DropPhase = 'WARMUP' | 'AIRDROP_LIVE' | 'LOOTED_OUT' |
'EXPIRED';

export interface PubgCrateDrop {
  readonly id: string;
  readonly title: string;      // e.g. 'PVS Finals VIP Pass + 5,000 G-Coin'
  readonly priceVnd: number;   // Formatted via Intl.NumberFormat('vi-VN')
  readonly remainingStock: number;
  readonly dropEpochMs: number;
}

export interface PubgVnSquadStanding {
  readonly squadId: string;    // 'te' | 'ces' | 'ge' | 'agg'
  readonly squadName: string;
  readonly wwcdCount: number;  // Winner Winner Chicken Dinner!
  readonly placementPts: number;
  readonly killPts: number;
}
```

HOMEWORK 3 ARCHITECTURE

HW 3 Code Contract: Anti-Drift Airdrop Hook & Optimistic Claim

Connecting Lab 2's createAntiDriftTimer to React 19 lifecycle hooks and conflict rollback.

Solving Timer Drift & Inventory Desync

- **Why setInterval(s ⇒ s - 1, 1000) Fails in PUBG Drops:** Gamers keep 10 browser tabs open waiting for G-Coin drops. Chrome throttles background tabs to 1 tick/min, causing naive timers to lag by minutes!
- **Monotonic Target Epoch + visibilitychange:** Compute Math.max(0, targetEpochMs - (Date.now() + serverOffsetMs)) on animation/interval frames AND immediately on tab wake.
- **Idempotent Optimistic Inventory Lock:** Disable duplicate clicks via an in-flight claimId lock, decrement stock optimistically, and rollback state if server returns 409 Looted Out.
- **Mandatory AI_FAILURE_AUDIT.md (15% Weight):** Document at least 3 AI bugs caught while building your KRAFTON PUBG VN Arena.

src/hooks/usePubgAirdropTimer.js

```
import { useEffect, useState } from 'react';

// [OK] EVOLVED FROM LAB 2 HW3 (createAntiDriftTimer) FOR REACT 19:
export function usePubgAirdropTimer(targetEpochMs, serverOffsetMs = 0) {
  const calcRemaining = () =>
    Math.max(0, targetEpochMs - (Date.now() + serverOffsetMs));

  const [remainingMs, setRemainingMs] = useState(calcRemaining);

  useEffect(() => {
    const syncClock = () => setRemainingMs(calcRemaining());
    syncClock();
    const timerId = window.setInterval(syncClock, 200);
    const onWake = () => {
      if (document.visibilityState === 'visible') syncClock();
    };
    document.addEventListener('visibilitychange', onWake);
    return () => {
      window.clearInterval(timerId);
      document.removeEventListener('visibilitychange', onWake);
    };
  }, [targetEpochMs, serverOffsetMs]);

  return { remainingMs, isLive: remainingMs === 0 };
}
```

APPLIED CREATIVE EXERCISE (V-POP FSM)

Bonus Exercise: 'Hẹn Hò Nhưng Không Yêu' State Machine

Inspired by Wendy Thảo's hit song: Solving 'Situationship' Boolean Soup & Asymmetric State.

Engineering Spec: <SituationshipAnalyzer />

- **1. Banning 'Situationship' Boolean Soup:** Why isDating=true && isLoved=false && hasTitle=false ('Hẹn hò nhưng không yêu') breaks UI invariants! Refactor into a strict FSM: 'STRANGERS' | 'DATING_NO_LOVE' | 'COMMITTED' | 'SELF_RESPECT_EXIT'.
- **2. Asymmetric Effort Reducer ('Cố Trăm Lần vs Một Phần'):** Model 'Em cố trăm lần (100x weight 0.1) chẳng bằng ai đó một phần (1x weight 50)' using immutable action logs and pure Derived State.
- **3. 'Seen Không Rep' Ghosting Timer & AbortController:** Sending a message starts a 5s reply timer in useEffect. If ignored or switched to 'Ai Đó', abort pending promises and trigger Garbage Collection!
- **4. Accessible Red-Flag Alert & 375px UI:** Bind all signal inputs with useId() and announce Red-Flag warnings via <output role='status' aria-live='polite'>.

hen-ho-khong-yeu-spec.ts

```ts
// [BAD] BOOLEAN SOUP BUG ('Hẹn hò nhưng không yêu, chắc em là kẻ mất trí'):
const [isDating, setIsDating] = useState(true);
const [isLoved, setIsLoved] = useState(false); // Contradictory UI!

// [OK] DETERMINISTIC FINITE STATE MACHINE CONTRACT:
export type RelationshipPhase =
  | 'FRIENDS'
  | 'DATING_NO_LOVE'    // 'Hẹn hò nhưng không yêu'
  | 'OFFICIAL_LOVE'     // Verified mutual commitment
  | 'GHOSTED_EXIT';     // Unmounted & memory freed

// Asymmetric Effort Formula ('Em cố 100 lần < Ai đó 1 phần'):
export interface EffortSignal {
  readonly id: string;
  readonly actor: 'EM' | 'AI_DO';
  readonly steps: number;      // e.g. 1000 steps vs 1 step
  readonly multiplier: number; // 0.05 vs 100.0
}
```

CREATIVE EXERCISE CODE CONTRACT

Reference Contract: SituationshipAnalyzer.jsx

Implementing immutable effort logs, derived Red-Flag warnings, and leak-free Ghosting cleanup.

Key React 19 Patterns Demonstrated

- **Pure Derived Compatibility Score:** Computes emScore, rivalScore, and redFlagWarning directly during render without a redundant useEffect.
- **Ghosting Timeout with Deterministic Cleanup:** When phase === 'DATING_NO_LOVE', a 5-second timer checks for mutual response; unmounting or clicking 'Dừng Lại' clears the timer cleanly.
- **Immutable Signal Dispatching:** Appends each interaction ({ id: crypto.randomUUID(), type, impact }) using spread syntax [...prev, signal].
- **Atomic Git Commits:** Commit 1: 'feat(fsm): model HenHoKhongYeu state machine' | Commit 2: 'feat(ui): add derived red-flag meter & ghosting cleanup'.

src/components/SituationshipAnalyzer.jsx

```
import { useEffect, useId, useState } from 'react';

export function SituationshipAnalyzer() {
  const inputId = useId();
  const [phase, setPhase] = useState('DATING_NO_LOVE');
  const [myTries, setMyTries] = useState(100); // 'Em cố trăm lần'
  const [rivalTries, setRivalTries] = useState(1); // 'Ai đó một phần'

  // [OK] DERIVED STATE: Purely computed during render
  const myImpact = myTries * 0.1;
  const rivalImpact = rivalTries * 50.0;
  const isRedFlag = phase === 'DATING_NO_LOVE' && myImpact < rivalImpact;

  useEffect(() => {
    if (phase !== 'DATING_NO_LOVE') return undefined;
    // Auto-exit situationship after 5s of unreciprocated ping (No Memory Leak!)
    const timerId = window.setTimeout(() => setPhase('GHOSTED_EXIT'), 5000);
    return () => window.clearTimeout(timerId);
  }, [phase, myTries]);

  return (
    <article aria-label='Hẹn Hò Nhưng Không Yêu Analyzer'>
      <output role='status' aria-live='polite'>
        {isRedFlag ? '[RED-FLAG] Red Flag: Hẹn hò nhưng không yêu — Giải phóng bộ
nhớ!' : '[OK] Stable'}
      </output>
      <button type='button' onClick={() => setPhase('GHOSTED_EXIT')}>Unbind & Free
Memory</button>
    </article>
  );
}
```

ARCHITECTURAL SYNTHESIS

Complete Continuity Matrix: Lab 2 Engine vs Lab 3 React 19

How every concept from Lab 2 maps directly into your Lab 3 engineering toolkit.

Lab 2 → Lab 3 Progression Summary

- **DOM Mounting: renderToDOM → createRoot:** Lab 2's synchronous recursive DOM creation becomes React 19's concurrent Fiber tree mounting in 'react-dom/client'.
- **State Memory: stateStore[cursor] → Fiber Hooks:** Lab 2's linear closure array becomes per-component Fiber hook queues (useState, useEffect, useId, useRef).
- **Performance: Custom memo() → React.memo:** Lab 2's shallowEqual wrapper powers Lab 3 HW 2's 88-key polyphonic piano so single keypresses never re-render 88 components.
- **Async Systems: Lab 2 Queues → KRAFTON PUBG VN:** Lab 2's ViewState<T>, AudioTaskQueue, and createAntiDriftTimer power Lab 3's PVS Telemetry and Airdrop Arena.

continuity-verification.json

```
{
 "Sprint_1": {
  "Lab2_Origin": "Sprint 1 VNode & Zero Div-Soup",
  "Lab3_Upgrade": "Semantic <DeveloperCard /> + CSS Modules"
 },
 "Sprint_2": {
  "Lab2_Origin": "Sprint 2 useState & O(n) Key Diffing",
  "Lab3_Upgrade": "ScientistDirectory Derived State (.toSorted)"
 },
 "Sprint_3_&_HW3": {
  "Lab2_Origin": "Sprint 3 ViewState & HW3 Anti-Drift Timer",
  "Lab3_Upgrade": "KRAFTON PUBG VN Telemetry & Airdrop Arena"
 }
}
```

GRADING & EVALUATION RUBRIC

100% Engineering Grading Rubric & Quality Gate

Identical assessment weights across Lab 2 and Lab 3 for rigorous competency verification.

The 5 Grading Dimensions (100% Total)

- **30% — Functional Correctness & State Integrity:** All 3 In-Class Sprints and 3 Homeworks (Drum Kit, 88-Key Piano, KRAFTON PUBG VN Arena) handle edge cases and immutable state cleanly.
- **25% — 3-Minute Live Refactoring Defense:** 1-on-1 oral defense at the grading desk (60–180s live code mutation with all AI tools closed) + deep explanation of Lab 2 → Lab 3 internals.
- **15% — Responsive 375px & WCAG 2.2 AA Compliance:** Zero horizontal scroll at 375px; zero soup; 100% form controls paired via <label htmlFor> + useId(); 100 Lighthouse A11y score.
- **15% — Code Quality, Modern Standards & Security:** Zero legacy APIs (no Class components, ReactDOM.render, defaultProps, keyCode, or alert()); zero XSS vectors.
- **15% — AI Failure Mode Audit & Atomic Git Commits:** Includes project-rules.md, TASK_DECOMPOSITION.md, >= 6–8 atomic commits, and forensic AI_FAILURE_AUDIT.md with 3 verified bugs.

grading-rubric-verify.sh

```
# FINAL SUBMISSION CHECKLIST BEFORE GRADING DESK:
├── project-rules.md          # AI constraints (React 19, Zero Div-Soup)
├── TASK_DECOMPOSITION.md     # WBS Sub-tasks mapped to Git
commits
├── AI_FAILURE_AUDIT.md       # >= 3 AI bugs + DevTools proof + fixes
├── src/
│   ├── components/
│   │   ├── DeveloperCard.jsx        # Sprint 1 (Zero <div>)
│   │   ├── ScientistDirectory.jsx   # Sprint 2 (Derived State)
│   │   └── PubgMatchTelemetry.jsx   # Sprint 3 (KRAFTON PUBG VN)
│   └── homeworks/
│       ├── DrumMachine/             # HW 1 (WCAG 2.2 AA Drum Kit)
│       ├── GrandPiano88/            # HW 2 (88-Key Polyphonic FIFO)
│       └── KraftonPubgVnArena/      # HW 3 (PVS Airdrop & Drops)
└── git log --oneline         # Minimum 6-8 atomic commits!
```

Generate with AI. Verify with Engineering.

From Lab 2 Mini-React VDOM to Lab 3 Production React 19 & KRAFTON PUBG VN Telemetry.

UIT - VNUHCM • Smart Software System Team (S3T) • MSc. Trần Vĩnh Khiêm